```
  <input id="contact_detail_phone_number" name="contact_detail[phone_number]" type="text" />
</form>
```

The object yielded by `fields_for` is a form builder like the one yielded by `form_for` (in fact `form_for` calls `fields_for` internally).

2.3 Relying on Record Identification

The Article model is directly available to users of the application, so - following the best practices for developing with Rails - you should declare it **a resource**:

```
resources :articles
```

Declaring a resource has a number of side-affects. See Rails Routing From the Outside Infor more information on setting up and using resources.

When dealing with RESTful resources, calls to `form_for` can get significantly easier if you rely on**record identification**. In short, you can just pass the model instance and have Rails figure out model name and the rest:

```
## Creating a new article
# long-style:
form_for(@article, url: articles_path)
# same thing, short-style (record identification gets used):
form_for(@article)

## Editing an existing article
# long-style:
form_for(@article, url: article_path(@article), html: {method:
"patch"})
# short-style:
form_for(@article)
```

Notice how the short-style `form_for` invocation is conveniently the same, regardless of the record being new or existing. Record identification is smart enough to figure out if the record is new by asking `record.new_record?`. It also selects the correct path to submit to and the name based on the class of the object.

Rails will also automatically set the `class` and `id` of the form appropriately: a form creating an article would have `id` and `class` `new_article`. If you were editing the article with id 23, the `class` would be set to `edit_article` and the id to `edit_article_23`. These attributes will be omitted for brevity in the rest of this guide.

When you're using STI (single-table inheritance) with your models, you can't rely on record identification on a subclass if only their parent class is declared a resource. You will have to specify the model name, `:url`, and `:method` explicitly.

2.3.1 Dealing with Namespaces

If you have created namespaced routes, `form_for` has a nifty shorthand for that too. If your application has an admin namespace then

```
form_for [:admin, @article]
```

will create a form that submits to the `ArticlesController` inside the admin namespace (submitting to `admin_article_path(@article)` in the case of an update). If you have several levels of namespacing then the syntax is similar:

```
form_for [:admin, :management, @article]
```

For more information on Rails' routing system and the associated conventions, please see the routing guide.